

Projeto de Testes de Software: Aplicando TDD na Camada de Negócio

1. Contexto do Problema

Uma clínica veterinária deseja criar um sistema simples para gerenciar o atendimento de animais e calcular os valores dos serviços prestados. O sistema precisa registrar animais, seus responsáveis e os atendimentos realizados, aplicando regras de negócio que podem evoluir no futuro. O foco do projeto é permitir que os alunos pratiquem TDD (Test-Driven Development), garantindo qualidade na implementação de métodos da camada de negócio, sem necessidade de banco de dados ou interface gráfica.

2. Objetivo do Projeto

- Aplicar TDD, criando testes antes da implementação.
- Implementar testes unitários focados na lógica de negócio.
- Desenvolver métodos de domínio coerentes e testáveis.
- Manipular estruturas de dados em memória, como listas de animais e atendimentos.
- Compreender a importância da manutenção da qualidade e refatoração segura baseada em testes automatizados.

3. Descrição do Sistema

O sistema deve gerenciar animais de uma clínica veterinária e os serviços realizados em seus atendimentos. Cada atendimento possui um serviço principal, podendo receber descontos, acréscimos ou regras específicas.

Regras básicas:

1. Consulta de rotina custa R\$ 100,00.
2. Consulta de urgência custa R\$ 180,00.
3. Atendimento de emergência custa R\$ 250,00.

Funcionalidades do sistema:

- Registrar um atendimento e calcular seu valor.
- Consultar o total gasto em atendimentos de um animal.
- Aplicar desconto de fidelidade de 10% quando o animal atingir pelo menos 5 atendimentos anteriores.
- Operar sobre listas de animais: adicionar, filtrar, ordenar, buscar e remover.

4. Escopo Técnico

- Sem interface gráfica.
- Sem banco de dados; todos os dados ficam em memória.
- Linguagem livre (Python, Java, C#, etc.) com biblioteca de testes (pytest, JUnit, NUnit, Jest).
- Estruturas de dados: listas e objetos simples.

5. Etapas do Desenvolvimento (Aplicando TDD)

1. Escrever o teste primeiro (ex.: calcular o valor de uma consulta de rotina).

2. Executar o teste (falha inicial).
3. Implementar o código mínimo para o teste passar.
4. Refatorar mantendo os testes verdes.
5. Repetir o ciclo para cada nova regra de negócio, incluindo operações sobre listas.

6. Exemplos de Testes a Criar

Nº	Nome do Teste (assinatura do método)	Objetivo / Cenário
1	test_calcular_valor_consulta_rotina()	Verificar se uma consulta de rotina custa R\$ 100,00.
2	test_calcular_valor_consulta_urgencia()	Confirmar que uma consulta de urgência custa R\$ 180,00.
3	test_calcular_valor_atendimento_emergencia()	Validar que um atendimento de emergência custa R\$ 250,00.
4	test_acumular_valores_varios_atendimentos()	Testar o acúmulo de valores em vários atendimentos consecutivos.
5	test_consultar_total_gasto_animal_existente()	Verificar se a consulta retorna o total correto gasto pelo animal.
6	test_aplicar_desconto_fidelidade()	Garantir desconto de 10% quando o animal possuir pelo menos 5 atendimentos anteriores.
7	test_nao_aplicar_desconto_sem_fidelidade()	Certificar que o desconto não seja aplicado antes do requisito mínimo.
8	test_calcular_atendimento_com_desconto()	Validar o valor final do atendimento após a aplicação do desconto.
9	test_nao_permitir_valor_servico_zero()	Assegurar que um serviço com valor zero seja rejeitado quando informado manualmente.
10	test_calcular_valores_decimais()	Confirmar que valores adicionais decimais sejam calculados corretamente.
11	test_nao_permitir_valor_negativo()	Garantir que o valor final de um atendimento nunca seja negativo.
12	test_animal_inexistente_lanca_excecao()	Verificar se o sistema lança erro ao consultar animal inexistente.
13	test_registrar_novo_animal_sem_atendimentos()	Validar o cadastro de um animal inicialmente sem histórico de atendimentos.
14	test_aplicar_acrescimo_procedimento_adicional()	Testar a inclusão de um procedimento adicional no valor do atendimento.
15	test_identificar_retorno_dentro_do_periodo()	Simular um retorno dentro de período determinado e aplicar a regra definida.
16	test_registrar_varios_animais_em_lista()	Validar a inserção de múltiplos animais em uma lista.
17	test_calcular_total_gasto_lista_animais()	Calcular os gastos acumulados de todos os animais de uma lista.
18	test_filtrar_animais_com_gasto_acima_de_limite()	Filtrar animais cujo total gasto seja superior a determinado valor.
19	test_ordenar_animais_por_total_gasto()	Ordenar animais conforme o total gasto acumulado.
20	test_remover_animais_sem_atendimentos()	Remover da lista animais que não possuem atendimentos registrados.
21	test_buscar_animal_por_nome()	Pesquisar animal pelo nome em uma lista.
22	test_somar_faturamento_total_lista()	Calcular o valor total dos atendimentos de todos os animais da lista.
23	test_ranking_animais_por_total_gasto()	Gerar ranking dos animais ordenado pelo total gasto em ordem decrescente.

7. Entregáveis

1. Código-fonte completo da aplicação.
2. Conjunto de testes unitários (cobrindo regras de negócio e operações em listas).
3. Relatório breve (máx. 4 páginas) contendo:
 - Estratégia de TDD utilizada.

- Casos de teste criados.
- Cobertura de código alcançada.

Obs: O trabalho, em dupla, tem valor 6,0 na Unidade I e a entrega deve ocorrer até 11 de setembro, 23h59, via Classroom. A partir desta data, em cada semana posterior de atraso na entrega, há uma penalização de 2 pontos.

8. Critérios de Avaliação

Aplicação correta do TDD - 30%

Qualidade e clareza dos testes - 25%

Coerência da lógica de negócio - 25%

Estrutura e organização do código - 10%

Relatório e documentação - 10%

9. Extensões Opcionais (para alunos avançados)

- Adicionar novas categorias de atendimento ou serviços veterinários.
- Criar regras de retorno, desconto e acréscimo mais complexas.
- Desenvolver operações adicionais sobre listas (ranking dinâmico, busca por faixa de gastos, merge de listas).

10. Linguagens/Bibliotecas Sugeridas

Python -> pytest, unittest

Java -> JUnit 5

C# -> xUnit, NUnit

JavaScript -> Jest